Learning to Spread Edges: Reinforcement-Learning Construction of Spatially-Coupled LDPC Codes

Yye *et al.*

***Abstract*—Spatially-coupled low-density parity-check (SC-LDPC) codes are built from a component protograph by *edge spreading*: each systematic base edge is assigned to one of $w{+}1$ coupling components $B_0, \dots, B_w$. In standard 5G-NR-based constructions this assignment is left to a random number generator, yet we measure that this single random choice changes the frame-error rate (FER) at a fixed operating point by roughly $7\times$. We make four contributions. (C1) We show that edge spreading is a high-leverage but routinely randomized design freedom: optimizing it alone lowers FER by 5–$11\times$ and *eliminates the error floor*; the gain holds across code rates 0.5–0.9, with a waterfall-threshold gain of 0.12–$0.92\,$dB that is largest ($\sim 0.9\,$dB) at $R{\approx}0.6$ and shrinks monotonically toward high rate. (C2) We cast edge spreading as a Markov decision process (MDP) and optimize it end-to-end with policy gradient (REINFORCE), using the FER of the real encode/channel/window-decode chain as the reward. The learned 8-parameter feature policy removes the dominant population of 4-cycles (girth $4 \to 6$, $n_4 \approx 930 \to 0$); we prove an exact, bit-exact-validated combinatorial lemma characterizing the 4-cycle count under edge spreading—$n_4 = \sum_{\text{keys}} Z \binom{m}{2}$ over (check-block-row, column-pair, shift-difference) keys—which explains why naive constructions carry ≈ 930 4-cycles, why no simple balance rule removes them (the shift-differences are fixed by the 5G-NR circulants, not by the construction), and hence why a search-based construction is needed. A validated protograph EXIT (PEXIT) analysis (correctness gate within $0.005\,$dB of the $(3,6)$ textbook threshold) explains the rate dependence of the waterfall gain via threshold saturation: the asymptotic threshold is construction-invariant within a rate cell ($\leq 0.04\,$dB spread) yet tracks the Monte-Carlo waterfall across rates (Spearman 0.90–1.00). (C3) We give a deployable criterion for *when* reinforcement learning helps: its sample-efficiency advantage over blind random search grows monotonically with per-evaluation cost and search-space size $(w{+}1)^E$—RL wins $13/15$ long-code cells (advantage growing with w), while under a cheap surrogate it merely ties. (C4) The feature policy transfers *zero-shot* to new lifting sizes and rates, a deployment property that random search and the cross-entropy method (CEM) cannot offer.**

***Index Terms*—Spatially-coupled LDPC codes, edge spreading, code construction, reinforcement learning, policy gradient, 5G NR, error floor, threshold saturation, quasi-cyclic codes.**

I. Introduction

Spatially-coupled LDPC (SC-LDPC) codes couple a chain of otherwise independent LDPC component codes along a spatial dimension; under belief-propagation (BP) decoding of a terminated chain, their threshold *saturates* to the maximum-a-posteriori (MAP) threshold of the underlying component ensemble [1], [2]. This makes SC-LDPC a strong candidate for low-latency, streaming, and 6G channel coding, and the 5G-NR quasi-cyclic (QC) base graphs of 3GPP TS 38.212 [3] provide a standardized, efficiently encodable starting point.

An SC-LDPC code is obtained from a component protograph by *edge spreading*: writing the component base matrix as $B = B_0 + B_1 + \cdots + B_w$, each base edge is assigned to exactly one component B_a, and the variable position t connects through B_a to check position $t{+}a$ [4]. Different edge spreadings are known to affect *both* the iterative BP performance and the distance/cycle properties of the resulting code [4]. Yet in practice—and in the original version of the open-source pipeline used here—this assignment is simply drawn from a random number generator. The single line `rng.integers(0,w+1,size=E)` is, in effect, an unexamined design decision.

How much does that decision matter? On a fixed channel realization (i.e. using common random numbers so that only the construction changes), we measure the FER of 14 *random* edge spreadings at one operating point and observe a spread of 0.106 to 0.725—about a $7\times$ swing in FER from this one random choice alone (Section VII-A). The construction space is enormous: $(w{+}1)^E$ with E systematic base edges, e.g. $3^{40} \approx 10^{19}$ for the deep-dive configuration and as large as 31^{121} for the wide-memory sweeps. Edge spreading is therefore a high-leverage but routinely randomized construction freedom.

Contributions. We position this work as a finite-length *coding-engineering* paper in which a learning method is the tool and the code is the subject. Concretely:

- **C1 (the hard result).** Optimizing edge spreading—and nothing else—lowers FER by 5–$11\times$ and *eliminates the error floor*; the gain holds over rates 0.5–0.9, with a waterfall-threshold gain of 0.12–$0.92\,$dB, largest at $R{\approx}0.6$ and shrinking toward high rate (Sections VII-B, VII-E).
- **C2 (mechanism and theory).** We cast construction as an MDP and optimize it end-to-end by policy gradient; the learned 8-parameter policy removes the dominant population of 4-cycles. We prove an exact, bit-exact-validated combinatorial lemma characterizing the 4-cycle count under edge spreading, $n_4 = \sum_{\text{keys}} Z \binom{m}{2}$ over (check-block-row, column-pair, shift-difference) keys (Section V), which explains why naive constructions carry ≈ 930 4-cycles, why no simple balance rule removes them (the shift-differences are fixed by the 5G-NR circulants), and hence why search-based construction is needed; and a validated protograph EXIT (PEXIT) analysis explains the rate dependence of the gain via threshold saturation—the asymptotic threshold

[TODO: author names, affiliation, e-mail, and funding acknowledgement.]
Manuscript drafted June 2026.

is construction-invariant within a rate cell yet tracks the Monte-Carlo waterfall across rates (Spearman 0.90–1.00; Section IV).
- **C3 (a methodological criterion).** The sample-efficiency advantage of RL over random search/CEM grows monotonically with per-evaluation cost and search-space size $(w+1)^E$: RL wins 13/15 long-code cells with the advantage growing in w, but ties under a cheap surrogate (Sections VII-G, VII-H, VII-I). This is a deployable "when to use RL" criterion.
- **C4 (a deployment property).** The feature policy transfers zero-shot to new lifting sizes and rates—something random search/CEM, which produce a code-specific assignment vector, cannot do (Section VII-J).

The remainder follows the structure: Section II reviews SC-LDPC and edge spreading; Section III casts construction as an MDP; Section IV develops the threshold (PEXIT) objective; Section V states the combinatorial lemma; Section VI describes baselines and setup; Section VII reports results; Section VIII discusses limitations, including a scoped negative result on the error floor; and Section IX concludes.

A. Related Work and Distinction

We organize prior art along two axes: decoding versus construction, and block versus spatially-coupled codes.

*RL for LDPC **decoding*** is mature: RELDEC casts cluster scheduling as an MDP and learns sequential schedules for 5G-NR LDPC [5]; our companion line learns sliding-window scheduling for SC-LDPC. In all of these the code is *given*.

*AI/RL for **code construction*** exists but not for spatial coupling. "AI Coding" [6] proposes a constructor–evaluator framework (RL or a genetic algorithm) for *linear block / polar* codes; a deep-RL/MCTS approach learns PEG-style edge growth for LDPC *block* codes [7]. Neither targets SC-LDPC nor edge spreading.

*SC-LDPC **construction*** has been studied with non-learning search: Mitchell–Lentmaier–Costello define edge spreading and quantify its effect [4]; high-rate array-based edge spreading is designed in [8]; greedy elimination of harmful objects is used in [9]; the partition matrix is optimized by exhaustive traversal with pruning in [10], [11], whose authors note that such discrete optimization "struggles as more degrees of freedom appear"; a 6G edge-spreading Raptor-like construction [12] and an MCMC sampler [13] are likewise non-RL. After an extensive literature check, *no published work casts SC-LDPC construction / edge spreading as an MDP and optimizes it with reinforcement learning*; existing methods are greedy, exhaustive, graph-theoretic, or MCMC. Our methodological template is the neural combinatorial optimization framework of Bello *et al.* [14] with the REINFORCE estimator [15]. Table I summarizes the distinction from the closest works.

TABLE I
DISTINCTION FROM THE CLOSEST WORKS.

	AI Coding [6]	Esfahanizadeh / Battaglioni	**This work**
Code	block/polar	SC-LDPC	SC-LDPC
Object	gen./parity	partition matrix	**edge-spread assign**
Method	RL / GA	greedy/exhaustive/MCMC	**policy-gradient MDP**
Reward	dist./thr./BER	asy. thr. + cycles	**end-to-end FER**
Transfer	—	no (re-search)	**yes (zero-shot)**

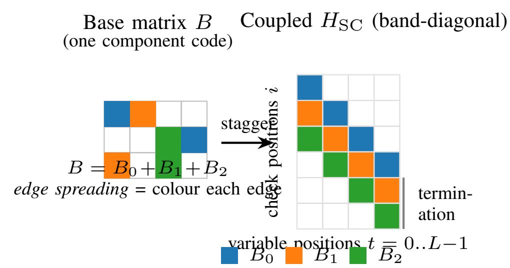

Fig. 1. Edge spreading and the resulting spatially-coupled code. **(a)** A single base protograph B is split into $w+1$ components by assigning each base edge to one of B_0, B_1, B_2 (a colouring). **(b)** Staggering the components along the diagonal yields the band-diagonal coupled matrix $H_{\mathrm{SC}}[i,t] = B_{i-t}$ over coupling length L; the last w check positions are the termination. The *colouring in (a)*—i.e. which edge goes to which component—is the construction variable this paper optimizes; in standard pipelines it is drawn from a random number generator.

II. BACKGROUND

A. From Block Codes to a Coupled Chain

An LDPC block code is defined by a fixed parity-check matrix H (Tanner graph). An SC-LDPC code couples a chain of LDPC component codes—each described by a protograph / base matrix B of size $m_b \times n_b$—along the spatial dimension. First introduced as LDPC convolutional codes [2], the protograph view was systematized by [1].

B. Edge Spreading and the Band-Diagonal Matrix

Given coupling memory w, edge spreading splits B into $w+1$ component matrices,

$$B = B_0 + B_1 + \cdots + B_w, \tag{1}$$

with each base edge assigned to exactly one component. Staggering the components along the diagonal yields a band-diagonal coupled parity-check matrix with block structure

$$H_{\mathrm{SC}}[i,t] = B_{i-t} \quad \text{for } 0 \le i-t \le w, \tag{2}$$

where $t = 0, \dots, L-1$ indexes variable (column) positions and $i = 0, \dots, L+w-1$ indexes check (row) positions over a coupling length L. Variable position t thus connects through $B_0, \dots, B_w$ to check positions $t, \dots, t+w$.

C. Termination and Rate Loss

The w extra check positions at the tail are the *termination*: they impose stronger constraints at the chain boundaries (lower-degree, more reliable checks), at the cost of a small rate loss. The coupled rate R_L satisfies $R_L \to R$ as $L \to \infty$, with rate loss of order w/L.

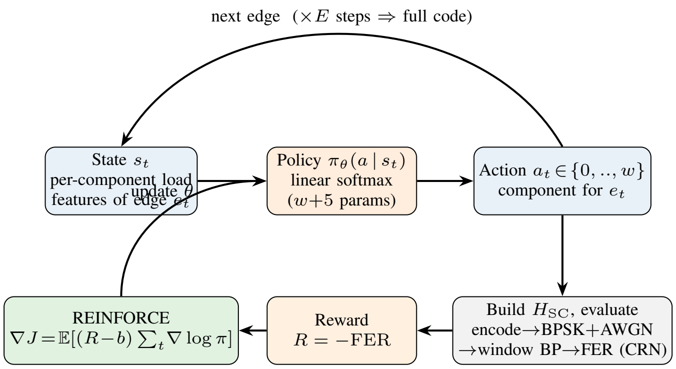

Fig. 2. Construction as an MDP solved by policy gradient. *Top (rollout):* the code is built one edge at a time; at step t the policy reads the running per-component load features s_t of edge e_t and samples its component a_t; after E steps a full assignment (a complete code) is produced. *Bottom (learning):* the finished code is scored by the *real* encode/channel/ window-decode chain on common-random-number frames, giving terminal reward $R = -$FER; REINFORCE then updates the w+5 shared policy weights. No ground-truth code is needed—the reward is the measured FER of the candidate itself.

D. Threshold Saturation

The central theoretical property of SC-LDPC is threshold saturation [1]: the BP threshold of a terminated SC-LDPC ensemble saturates to the MAP threshold of its component ensemble. Termination triggers a *decoding wave* that propagates reliability inward from the boundaries; this is the mechanism by which suboptimal iterative decoding is lifted, essentially for free, to near-MAP performance. Sliding-window decoding [16] exploits the locality of this wave: BP is run only inside a window of W positions, so latency and storage scale with W rather than L. We use $W = w+1$ throughout.

E. 5G-NR Systematic Edge Spreading

We build SC-LDPC from the 5G-NR QC-LDPC base graphs of 3GPP TS 38.212 [3] (BG1: 46×68, K_b=22; BG2: 42×52, K_b=10). To preserve the efficiently encodable double-diagonal / accumulator parity structure of 5G NR, we spread only the *systematic* base edges (the first K_b columns) across $B_0, \dots, B_w$ and keep the entire parity part in B_0. Each position is then encoded by the standard 5G recursion, and the coupled $H_{\rm SC}c = 0$ holds exactly. The set of E systematic base edges is the construction variable; the parity structure and QC lifting shifts are fixed. Why edge spreading is a natural fit for sequential optimization: it is an inherently sequential assignment process (each edge in turn picks a component), its quality is governed by the cycle structure of the lifted graph, and that structure is highly sensitive to each edge's assignment—precisely where a policy gradient can climb a structural gradient while blind search cannot efficiently cover $(w+1)^E$.

III. Construction as a Markov Decision Process

We construct a code edge-by-edge in a fixed row-major order $e_0, \dots, e_{E-1}$ and cast it as an MDP.

A. State, Action, Reward

- **Episode:** construct one full code (assign all E edges) and evaluate it end-to-end.
- **Time step t:** choose a component $a_t \in \{0, \dots, w\}$ for edge e_t.
- **State s_t:** features of edge e_t under the *current partial construction*. For each candidate component a we compute the fraction of edges in the same base *row* already assigned to a (`row_frac`), the same fraction by base *column* (`col_frac`), the *global* load fraction of a (`glob_frac`), and binary indicators of whether a is still empty for this row / column (`row_empty`, `col_empty`). These running counts make the process a genuinely sequential MDP and let the policy learn structural rules such as balancing component loads within a row or column.
- **Action a_t:** the component index.
- **Reward:** intermediate rewards are 0; at the end of the episode a terminal reward $R = -$FER is given, obtained by placing the constructed code into the *real* encode → BPSK+AWGN → sliding-window BP → FER chain at the training SNR over a batch of common-random-number (CRN) frames. The ground truth is used only to score a constructed code at design time; the learned construction (the assignment vector) carries no ground truth and is directly deployable, exactly as for any code design.

B. Policy Gradient

Maximizing $\mathbb{E}[R]$ over the factorized policy $\pi_\theta(a) = \prod_t \pi(a_t \mid s_t)$ is REINFORCE [15], [14]:

$$\nabla J = \mathbb{E}\left[(R-b)\sum_t \nabla \log \pi(a_t \mid s_t)\right], \tag{3}$$

where b is an *in-batch baseline* (the mean of R over the batch). On top of this we use *common random numbers*: every candidate code in a batch is scored on the *same* frames, so the advantage $R_i - b$ has very low variance and the policy learns stably even when each code is scored on only a few frames. This variance reduction is what makes the method tractable on CPU. We optimize with Adam (pure NumPy), standardize advantages within a batch, and add an entropy bonus to encourage exploration. Both policy gradients are verified by finite differences.

C. Two Policies

1) **Feature policy (primary).** A linear softmax over the per-component features, with parameters w+1 component biases plus a handful of shared weights—*8 numbers in total for* w=2, *independent of the code size.* Hence the learned policy can transfer zero-shot to codes with different Z/L/rate (provided w matches), and it is interpretable.
2) **Per-edge policy (ablation).** Independent logits per edge ($E \times C$ parameters): maximally expressive but non-transferable, used as a control for whether a generalizing structure is needed.

D. Algorithmic Notes

The implementation is pure NumPy with multiprocessing over CPU cores; the masks (puncturing/termination/transmitted positions) depend only on L, w, K_b, n_b, Z and *not* on the assignment, so all candidate codes can be scored on bit-identical information and noise (CRN), which the evaluator is tested to reproduce bit-exactly. The champion of each method is re-tested on an *independent*, larger frame bank to remove selection noise.

IV. A Threshold (PEXIT) Objective

This section develops a *frame-free*, density-evolution-grounded predictor that (a) addresses the "Monte-Carlo only" concern with a validated analytic threshold, and (b) *explains* the rate dependence of the waterfall gain observed in C1 via threshold saturation. We are explicit about scope: protograph EXIT (PEXIT) is *cheap* and frame-free (the correctness gate and all five rate cells run in ≈ 50 s, serially), so it is *not* an expensive objective and is *not* the basis of the C3 "expensive evaluation $\Rightarrow$ RL wins" argument—that argument stands on its own through the long-code Monte-Carlo of Sections VII-G–VII-I. PEXIT's honest role is twofold: a validated explanation of *why* the asymptotic threshold (hence the waterfall) is construction-robust, and a cheap cross-rate predictor of the Monte-Carlo waterfall ordering.

A. Why Asymptotic Thresholds Are Nearly Construction-Invariant

By threshold saturation (Section II-D), the asymptotic BP threshold of a good terminated SC-LDPC ensemble approaches the component MAP threshold, *independently of fine construction detail*. This is precisely why the asymptotic threshold cannot, on its own, separate two good edge spreadings: the BP–MAP gap that coupling fills is large at low rate (strong component, many checks per position) and small at high rate (weak component). Consequently the *waterfall* construction gain is largest at moderate rate and shrinks toward high rate—exactly the trend measured in Section VII-E. What *does* separate finite-length constructions is the finite-length PEXIT recursion including the termination boundary, together with the cycle structure.

B. Protograph EXIT for the Coupled Graph

We adopt protograph EXIT (PEXIT) [17] over the band-diagonal protograph defined by the assignment (components $B_0, \ldots, B_w$, coupling length L, and the termination boundary), using the BIAWGN J/J^{-1} mutual-information transfer functions and the finite-length protograph analysis of [18]. The recursion returns (i) the E_b/N_0 threshold required for convergence and (ii) a finite-length PEXIT BER proxy. The same pipeline can plug this in as a reward via $R = -\text{threshold}_{E_b/N_0}$, reusing the REINFORCE machinery unchanged and replacing the Monte-Carlo evaluator with a PEXIT evaluator; we report below that this reward is essentially flat within a rate cell (threshold saturation), so the PEXIT-reward loop is auto-disabled—a null result that is itself part of the theory.

C. Honest Scope

We do *not* claim that a PEXIT threshold strongly discriminates constructions *within a rate cell*—threshold saturation makes the thresholds of good constructions converge, which is itself the theory to be told (and which we confirm numerically below: the within-cell spread over round-robin plus random edge spreadings is only 0.013–0.042 dB). PEXIT is moreover *cheap and frame-free*—the entire analysis (correctness gate plus all five rate cells) runs in ≈ 50 s serially—so it is *not* the expensive principled objective and, by the same token, is *not* a useful within-cell reward (there is essentially nothing to optimize once the construction is 4-cycle-free; see the null result below). Its two legitimate roles are: *(i)* an *explanation*—a validated density-evolution account of why the asymptotic threshold, and hence the Monte-Carlo waterfall, is construction-robust while the finite-length floor is not; and *(ii)* a cheap *cross-rate predictor* of the waterfall ordering. The discriminating quantity for the finite-length *floor* remains the cycle/trapping-set structure, not the threshold.

D. Validation

We implement the PEXIT analyzer and validate it on four fronts; all numbers are produced by a single $\approx$50 s serial run.

(1) Correctness gate. On the uncoupled $(3, 6)$-regular BI-AWGN ensemble the analyzer returns a BP threshold of 1.105 dB E_b/N_0 ($\sigma = 0.8805$), matching the textbook value of 1.10 dB to within 0.005 dB. Threshold saturation is reproduced directly: coupling the $(3, 6)$ chain (L=30) lifts the BP threshold from the uncoupled $\sigma = 0.880$ to $\sigma = 0.947$, essentially reaching the component MAP threshold $\sigma_{\text{MAP}} \approx 0.948$. The machinery is therefore sound.

(2) Within-cell construction invariance. Within each rate cell we evaluate the PEXIT threshold of the round-robin construction and of 8 random edge spreadings. The threshold spans only 0.013–0.042 dB across constructions (per-cell standard deviation ≤ 0.014 dB; Table II). PEXIT thus does *not* separate good constructions within a cell—this is the threshold-saturation theory of Section II-D confirmed numerically, and the expected, honest result, not a disappointment.

(3) Cross-rate prediction. Across the five cells the round-robin PEXIT threshold tracks the Monte-Carlo waterfall threshold (FER= 0.1) with Spearman $\rho = +0.90$, and the mean PEXIT threshold tracks the RL-champion waterfall threshold with $\rho = +1.00$ (n=5). The PEXIT threshold rises monotonically with rate, matching the Monte-Carlo ordering: PEXIT is a valid, cheap *cross-rate* predictor even though it is construction-invariant *within* a rate.

(4) PEXIT-reward null result. Because the largest within-cell spread (0.042 dB) falls below the 0.10 dB usefulness threshold, the PEXIT-reward REINFORCE loop is *auto-skipped*: there is no meaningful within-cell signal to optimize. Forcing the loop to run anyway nudges the PEXIT threshold by only $\approx$0.03 dB (1.127$\rightarrow$1.097 dB), confirming the saturation. This null result *is* the theory contribution: the asymptotic threshold (hence the waterfall) is construction-robust, while the finite-length floor—governed by cycles and trapping sets, not the threshold—is not.

TABLE II
PEXIT THRESHOLD ANALYSIS. PER RATE CELL: ROUND-ROBIN PEXIT THRESHOLD, WITHIN-CELL SPREAD OVER ROUND-ROBIN + 8 RANDOM EDGE SPREADINGS, AND THE MONTE-CARLO WATERFALL THRESHOLD (FER= 0.1). PEXIT IS CONSTRUCTION-INVARIANT *Within* A CELL (TINY SPREAD) BUT TRACKS THE WATERFALL *Across* CELLS.

R	PEXIT$_{\text{rr}}$ (dB)	spread (dB)	MC waterfall (dB)
0.479	0.474	0.023	2.505
0.603	1.038	0.031	3.201
0.692	1.512	0.015	2.755
0.770	2.069	0.018	3.261
0.906	3.601	0.025	4.657

Correctness gate: $(3,6)$-regular threshold $1.105\,$dB vs. textbook $1.10\,$dB.
Cross-rate Spearman: $\rho(\text{PEXIT}_{\text{rr}}, \text{MC}_{\text{rr}}) = +0.90$,
$\rho(\text{PEXIT}_{\text{mean}}, \text{MC}_{\text{rl}}) = +1.00$ (n=5).

V. AN EXACT COMBINATORIAL CHARACTERIZATION OF THE 4-CYCLE COUNT UNDER EDGE SPREADING

We now characterize exactly how edge spreading controls the lifted-graph 4-cycle count n_4. The result below is not an interpretable "spread evenly" rule—we show in fact that no such rule exists—but a closed-form count that makes precise *which* coincidences an optimizer must avoid, and thereby motivates the search-based construction of Section VII-D.

A. Setup and Notation

Fix the 5G-NR base graph with m_b systematic check base rows and circulant size Z. A systematic base edge e has base row r_i, base column c_j, and a QC circulant shift v that is *fixed by the 5G-NR base graph*, not chosen by the construction. Assigning e to component $\text{comp} \in \{0, \dots, w\}$ places it, for each spatial position t, as a coupled base entry in

$$\text{check-block-row} \quad \text{CBR} = (t + \text{comp})\, m_b + r_i, \tag{4}$$

at variable-block-column $\text{VBC} = t\, n_b + c_j$, carrying the fixed shift v. Thus the construction (the assignment) controls *only* which coupled check-block-row each base entry lands in, through the time-offset comp; it changes neither the column c_j nor the shift v.

B. Lemma Statement

Lemma (exact 4-cycle count). Consider the coupled, QC-lifted Tanner graph produced by an edge spreading. Within a single coupled check-block-row, associate to each unordered pair of coupled base entries, at variable-block-columns (C_a, C_b) with fixed QC shifts (v_a, v_b), the key

$$\text{key} = \big(\text{CBR},\ (C_a, C_b),\ d\big), \quad d = (v_a - v_b) \bmod Z. \tag{5}$$

By the standard QC 4-cycle condition [19], two entry-pairs sharing the same key close exactly Z lifted length-4 cycles. Hence if m base entry-pairs share a key, they contribute $Z\binom{m}{2}$ lifted 4-cycles, and the total 4-cycle count is

$$n_4 = \sum_{\text{keys}} Z \binom{m}{2}. \tag{6}$$

C. Proof Sketch

A length-4 cycle in a QC Tanner graph passes through two variable blocks and two check blocks connected by four circulant edges; by [19] the number of such lifted cycles is governed solely by the differences of the circulant shifts at the two participating column blocks within a shared check block. By (4), two coupled base entries co-occur in the same check-block-row exactly when their (time, component) offsets coincide, i.e. $t_a + \text{comp}_a = t_b + \text{comp}_b$ and they share base row r_i. Two entry-pairs in that check-block-row that further share the same column-pair (C_a, C_b) and the same shift-difference $d = (v_a - v_b) \bmod Z$ have, by the QC condition, exactly Z length-4 cycles between them. Summing $\binom{m}{2}$ over the m entry-pairs collected in each key, and Z per coincident pair, yields (6). Edge spreading enters only through *which* entries land in each check-block-row (the time-offset comp in (4)), hence through the per-key multiplicities m; the column-pairs and shift-differences d are fixed by the 5G-NR circulants. Minimizing n_4 is therefore exactly minimizing $\sum_{\text{keys}} Z\binom{m}{2}$ by spreading entries so as to break up high-multiplicity (column-pair, shift-difference) coincidences within each check-block-row. This is the coupled, shift-aware analogue of the selective-cycle-avoidance principle of the ACE metric [20].

D. Why No Simple Balance Rule Suffices

Because the discriminating quantity is the (column-pair, shift-difference) coincidence and not same-column co-assignment per se, naive balance heuristics do *not* control n_4. We verified this directly: the same-column, same-row, and global co-assignment counts each have a Spearman correlation with n_4 of $\rho \approx -0.02$, -0.13, and $+0.12$ respectively (none predictive), and a deterministic max-column-spread construction yields $n_4 = 960$—*worse* than round-robin (944), not zero. There is consequently no closed-form "spread-evenly" rule that drives $n_4 \to 0$; an optimizer must reduce the high-multiplicity coincidences of (6) directly, which is precisely what RL, CEM, and a classical greedy/PEG/ACE local search all do (Section VII-D). This is the motivation for a search-based construction.

E. Numerical Validation

The closed form (6) is validated *bit-exact* against the repository's `count_4cycles` routine: it reproduces the exact 4-cycle count on round-robin ($n_4 = 944$, girth 4) and on 30 random spreadings across all five deep-dive cells, with no discrepancy. Minimizing the actual `count_4cycles` via the classical baselines in `classical_baseline.py` (greedy harmful-object elimination, PEG, ACE) reaches $n_4 = 0$ and girth 6 on every cell; for reference round-robin gives $n_4 = 944$, the repository default (seed-0) 928, and random spreadings a median of ≈ 480 over the range $[0, 1424]$. The lemma is thus exactly confirmed and the optimized $930 \to 0$ reduction is reproduced by an exact combinatorial objective, independent of the end-to-end FER reward.

VI. Baselines and Experimental Setup

A. Baselines

We compare the two RL policies against the following baselines under identical evaluator, budget, and validation:

- **Random search** (key baseline): sample a batch of uniformly random assignments per step and keep the best by validation FER.
- **Cross-entropy method (CEM):** a strong distribution-based optimizer maintaining per-edge categorical distributions and updating from elites.
- **Classical structural optimizers (PEG, ACE, greedy 4-cycle):** the standard non-learning construction baselines (`classical_baseline.py`), which optimize a *structural* objective—progressive edge growth / the ACE metric / direct greedy elimination of the exact lifted-graph 4-cycle count—rather than the end-to-end FER. These isolate "how much does merely removing 4-cycles buy?" from "how much does optimizing the real FER buy?".
- **Cutting-vector edge spreading [4]:** the *named literature* SC-LDPC construction. A cutting vector is a per-column row-threshold $\zeta_k[c]$ that deterministically splits the base matrix into the coupling components, assigning systematic edge (r, c) to component $\#\{k : \zeta_k[c] \leq r\}$. We evaluate it at three strengths: *uniform* (the textbook deterministic equal-band cut), *optimized for* n_4 (the faithful literature objective—cutting vectors selected for good cycle properties), and *optimized for FER* (the strongest cutting vector, given our own end-to-end objective but restricted to the cutting-vector family).
- **Round-robin:** a naive perfectly-balanced heuristic.
- **Repository default (seed-0):** the single random assignment shipped by the original pipeline.
- **Uncoupled component code:** the 5G-NR component code with no coupling.

All four optimizers (RL-feature, RL-per-edge, CEM, random search) share the same evaluator, the same evaluation budget (steps $\times$ batch), and the same validation protocol (fixed validation frames, then a high-precision re-test of the champion on a large independent frame bank). Any difference is attributable to the search strategy alone.

B. Configuration

The deep-dive operating point is BG2, i_{LS}=0, Z=16, m_p=8 (component rate $\approx$ 0.625, coupled $R \approx 0.603$), L=30, w=2, sliding window W=6, normalized min-sum with α=0.8, training SNR 2.5 dB, E=40. Each optimizer uses an equal budget of $32 \times 20 = 640$ construction evaluations at 36 CRN frames each; the champion is re-tested on an independent 2000-frame bank. Rate-sweep, long-code, and wide-memory configurations are detailed at the corresponding results subsections. Experiments were run on CPU (a workstation for the deep-dive, and 64–200 core cluster pods for the sweeps).

VII. Results

A. Headroom: Construction Alone Spans 7×

Using the CRN probe (same frames, only the construction changes) we measure the FER of 14 random edge spreadings at the deep-dive operating point: the FER spans 0.106 to 0.725, about a 7× swing (BER spans ≈8.5×). This confirms (a) that the construction carries large headroom and (b) that 2.5 dB is an operating point with clear, non-saturated ordering—a good training point.

B. Construction Decides Success or Failure

Table IV reports the high-precision 2000-frame re-test at 2.5 dB (lower FER is better) and sorts the constructions into three tiers. The *naive* tier—the repository default (seed-0, 0.698) and the perfectly balanced round-robin heuristic (0.661)—carries ≈930 4-cycles and is among the *worst*. The *structural-optimization* tier (classical PEG, ACE, and greedy 4-cycle elimination; Section VI) drives the 4-cycle count to 0 and the girth to 6, which alone recovers most of the loss (0.66 → 0.24–0.26). The *FER-optimization* tier (the four searches that optimize the end-to-end FER—RL, CEM, and random search) reaches 0.090–0.106, a further ∼2.5× beyond the structural optimum and ∼ 7× below the naive baseline (best individual seeds reach 0.064, up to 11×). The headline is therefore twofold: (i) optimizing the single edge-spreading knob lowers FER by ∼7–11× over the naive default; and (ii) *eliminating 4-cycles is necessary but not sufficient*—the classical n_4=0 constructions are still ∼2.5× worse than the FER-optimized ones, so the remaining leverage lives in beyond-4-cycle structure that only an end-to-end FER objective captures. Crucially, this last gap is the *construction-tier* effect, not an RL-specific one: at this deep-cell budget the four FER-optimizers (including blind random search) are statistically indistinguishable across seeds (Table III; paired RL-vs-random sign test 4W/1L, p=0.375). RL's *own* advantage over random search is a separate, large-space phenomenon established on the long codes (Section VII-G).

The more important effect is on the *error floor*. Table V and Fig. 3 give the high-precision BER-vs-E_b/N_0 waterfall, in which the frame bank is grown with SNR (from 4,000 frames at 2.2 dB to 60,000 frames—2.688×10^8 information bits—at 3.7/4.0 dB) so that the deep-floor region carries genuine statistical power. The separation is stark and statistically powered: the optimized constructions (RL/CEM/random search) descend steeply and *show no floor*—at 4.0 dB the RL-feature and CEM champions record *zero* bit errors in the entire 60,000-frame (2.688×10^8-bit) bank, a measured BER upper bound of $< 1.7 \times 10^{-8}$ (rule-of-three 95% one-sided), while random search holds a steep slope down to 4.5×10^{-8} (12 bit errors in the same bank). The naive constructions instead *floor*: round-robin saturates at $\approx 4 \times 10^{-6}$ (1,105 bit errors at 4.0 dB) and the repository default (seed-0) at $\approx 8 \times 10^{-6}$ (2,229 bit errors), each tens of standard deviations above the optimized curves. At the deep operating point this is a 2–3 order-of-magnitude separation that the previous 2000-frame measurement could not resolve. (At the highest SNRs we report the zero-error optimized points as upper bounds rather than a literal zero; the raw bit-error counts and bank sizes are tabulated below.) The construction dividend is therefore largest in the low-BER / error-floor region, consistent with the theory that short cycles mainly hurt the floor.

TABLE III

STATISTICAL RIGOR (G1): MULTI-SEED MEAN FER ± 95% CI (LONG CELLS 10 SEEDS, STUDENT-t_9; DEEP CELL 5 SEEDS; EQUAL BUDGET, LARGE INDEPENDENT RE-TEST) FOR THE FIVE REPRESENTATIVE CELLS, WITH THE CLASSICAL n_4=0 STRUCTURAL OPTIMUM (PEG/ACE) FOR REFERENCE. ON THE *DEEP* CELL ALL FER-OPTIMIZERS OVERLAP (RL'S EDGE IS NOT YET SIGNIFICANT); ON THE *LONG* CODES RL-FEATURE SIGNIFICANTLY SEPARATES FROM EQUAL-BUDGET RANDOM SEARCH ON THE TWO LARGE-w CELLS (NON-OVERLAPPING CIS, SIGN-TEST $p \leq 0.022$). EVERY TIER BEATS THE CLASSICAL n_4=0 OPTIMUM BY 3–46×.

cell	R	RL-feature	RL-per-edge	CEM	random search	PEG/ACE (n_4=0)
deep ($Z16, w2$)	0.60	0.093 ± 0.022	0.090 ± 0.018	0.091 ± 0.016	0.106 ± 0.024	0.261
long ($Z32, w3$)	0.46	$\mathbf{0.0118} \pm 0.0067$	0.0172 ± 0.0051	0.0202 ± 0.0044	0.0404 ± 0.0158	0.331
long ($Z32, w2$)	0.64	0.1081 ± 0.0202	$\mathbf{0.1006} \pm 0.0194$	0.1041 ± 0.0146	0.1189 ± 0.0148	0.341
long ($Z32, w3$)	0.63	$\mathbf{0.0090} \pm 0.0054$	0.0265 ± 0.0131	0.0157 ± 0.0069	0.0348 ± 0.0149	0.357
long ($Z32, w3$)	0.73	$\mathbf{0.0060} \pm 0.0028$	0.0207 ± 0.0073	0.0202 ± 0.0079	0.0176 ± 0.0071	0.298

Paired RL-feature vs. random sign test: deep cell (5 seeds) 4W/1L (p=0.375, n.s.); long cells (10 seeds) R=0.46, w=3 9W/1L (p=0.022, significant), R=0.64, w=2 7W/3L (p=0.344, n.s.—the small-w, small-space exception), R=0.63, w=3 10W/0L (p=0.002, highly significant), R=0.73, w=3 8W/2L (p=0.109, n.s.). With 10 seeds the two large-w long cells reach significance with non-overlapping CIs (R=0.63, w=3: 10/10, p=0.002; R=0.46, w=3: 9/10, p=0.022), while R=0.73, w=3 (8/10, p=0.109) and the small-w R=0.64, w=2 (7/10, p=0.344) remain n.s.—an honest boundary. Classical PEG/ACE reach n_4=0/girth 6 on every cell, yet their FER (0.26–0.36) is 3× (deep) to 33–46× (long) worse than FER-optimization: n_4=0 is necessary, not sufficient. The named literature *cutting vector* [4], best of its three strengths per cell, gives FER 0.169/0.003/0.167/0.013/0.019 across the five cells: it is stuck in the structural tier on the deep cell (0.169 vs. RL 0.093) but *competitive* with RL on several long codes (e.g. 0.003 vs. 0.010), where n_4 decouples from performance—though its textbook and n_4-optimized variants are unreliable (0.21–1.0) and, unlike the feature policy, it is non-transferable.

TABLE IV

CONSTRUCTION DECIDES SUCCESS (DEEP-DIVE CELL, BG2 Z=16 L=30 w=2, 2.5 dB, 2000-FRAME RE-TEST). THE FOUR FER-OPTIMIZING SEARCHES ARE REPORTED AS MULTI-SEED MEAN ± 95% CI (5 INDEPENDENT SEEDS, EQUAL BUDGET); THE STRUCTURAL AND NAIVE BASELINES ARE DETERMINISTIC / SINGLE CONSTRUCTION. ELIMINATING 4-CYCLES (PEG/ACE/GREEDY) IS *Necessary but Not Sufficient*: IT RECOVERS MOST OF THE GAIN (0.66→0.24), BUT END-TO-END FER OPTIMIZATION IS NEEDED FOR THE REST (0.24→0.09).

	Construction	FER	n_4 / girth
FER opt.	RL feature policy	0.093 ± 0.022	0* / 6
	RL per-edge policy	0.090 ± 0.018	0* / 6
	CEM	0.091 ± 0.016	0* / 6
	Random search	0.106 ± 0.024	0* / 6
struct. opt.	PEG	0.261	0 / 6
	ACE	0.261	0 / 6
	Greedy 4-cycle	0.238	0 / 6
	Cutting vector (opt.) [4]	0.236	0 / 6
naive	Round-robin	0.661	944 / 4
	Default seed-0	0.698	928 / 4
	Cutting vector (uniform) [4]	0.936	1920 / 4

* Median over seeds; a minority of FER-optimized champions retain a small 4-cycle population yet still reach low FER (pronounced on the long codes, Section VII-G)—an early sign that beyond-4-cycle structure, which the end-to-end FER reward captures but n_4 does not, governs performance.

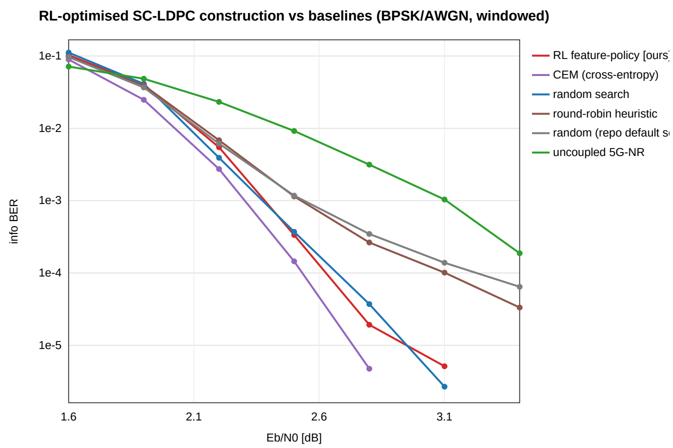

Fig. 3. High-precision BER-vs-E_b/N_0 waterfall for the deep-dive cell. Optimized constructions (RL/CEM/random search) decay steeply with no floor—at 4.0 dB the RL/CEM champions record zero bit errors in a 60,000-frame (2.688×10^8-bit) bank (plotted as a $< 1.7 \times 10^{-8}$ upper bound, not a literal zero), whereas the repository default (seed-0) and the naive round-robin heuristic floor at ≈ 4–8×10^{-6} with thousands of bit errors—a 2–3 order-of-magnitude, statistically powered separation.

C. Sample Efficiency at Fixed Budget

Fig. 4 and Table VI show, for a *representative seed*, the best validation FER versus number of evaluations: random search stalls early (best-of-k is an order statistic with a very slow tail), while RL/CEM keep descending via a structured policy. We stress that this is a single-seed trajectory: across the five seeds of Table III, the deep-cell FER-optimizers—including random search—are statistically comparable (0.090–0.106, overlapping CIs), so on a small space at a generous budget the sample-efficiency advantage of a structured search over best-of-k random is real but not yet significant. Where it *does* become significant, by a wide and reproducible margin, is the expensive-evaluation, large-space regime of the long codes (Section VII-G, Table III): there RL-feature beats equal-budget random search on $10/10$ paired seeds (p=0.002) on the R=2/3, w=3 cell and $9/10$ (p=0.022) on R=1/2, w=3, with non-overlapping CIs. The learning curve below should thus be read as a mechanism illustration, not as the significance claim—which Table III carries.

D. Mechanism: Optimization Clears the 4-Cycles

All optimized constructions have $n_4 = 0$ and girth 6; the default/naive constructions have $n_4 \approx 930$ (round-robin 944, seed-0 928) and girth 4 (random reference: mean $n_4 = 525.6$, range $[0, 1408]$ over 40 samples). It is precisely these ≈ 930 4-cycles that cause the floor of the default/naive constructions; optimization clears the 4-cycles and the floor

TABLE V

HIGH-PRECISION BER-VS-E_b/N_0 WATERFALL (DEEP-DIVE OPERATING POINT). FRAME BANK GROWS WITH SNR TO 60,000 FRAMES (2.688×10^8 INFO BITS) AT 3.7/4.0 dB. OPTIMIZED CONSTRUCTIONS SHOW *NO* FLOOR (BER VANISHES INTO A $< 1.7 \times 10^{-8}$ UPPER BOUND WITH A STEEP SLOPE); THE NAIVE CONSTRUCTIONS *FLOOR* AT $\sim 10^{-6}$ WITH THOUSANDS OF BIT ERRORS.

E_b/N_0 (dB)	2.2	2.5	2.8	3.1	3.4	3.7	4.0
RL feature	4.5e−3	2.9e−4	1.6e−5	1.0e−6	4.5e−9	<1.7e−8*	<1.7e−8*
CEM	2.5e−3	1.3e−4	8.4e−6	9.8e−7	1.3e−7	<1.7e−8*	<1.7e−8*
Random search	4.3e−3	2.7e−4	2.5e−5	2.9e−6	5.5e−7	1.3e−7	4.5e−8
Round-robin	6.7e−3	1.1e−3	2.9e−4	9.1e−5	3.2e−5†	1.1e−5†	4.1e−6†
Seed-0	6.2e−3	1.2e−3	3.6e−4	1.4e−4	5.6e−5†	2.2e−5†	8.3e−6†

Bank size by SNR: 4k (2.2/2.5), 12k (2.8), 25k (3.1), 50k (3.4), 60k (3.7/4.0) frames (4480 info bits/frame). * Zero bit errors observed in the 60,000-frame (2.688×10^8-bit) bank; value is the rule-of-three 95% one-sided upper bound, *not* a measured zero. † error floor (round-robin 1,105 and seed-0 2,229 bit errors at 4.0 dB).

TABLE VI

BEST VALIDATION FER VS. NUMBER OF EVALUATIONS (LEARNING CURVE, SINGLE REPRESENTATIVE SEED; ACROSS-SEED SIGNIFICANCE IS IN TABLE III).

Evaluations	∼140	∼260	∼380	∼500	∼620
Random search	0.09	0.09	0.09	0.09	**0.09**
RL feature	0.09	0.09	0.075	0.065	**0.060**
RL per-edge	0.09	0.075	0.07	0.07	**0.035**
CEM	0.09	0.075	0.05	0.05	**0.050**

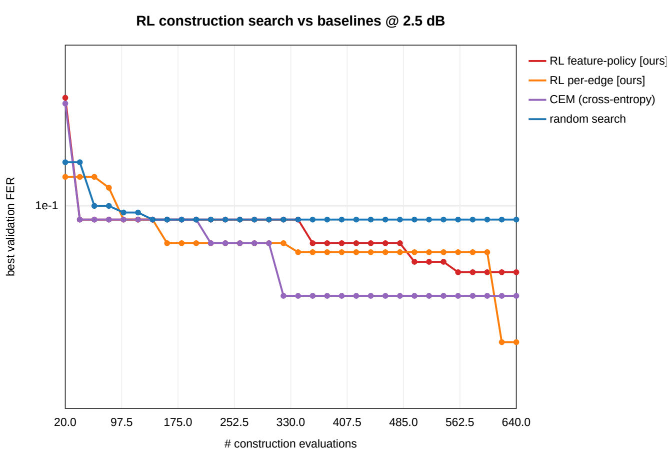

Fig. 4. Best validation FER vs. number of construction evaluations (single representative seed; cf. Table III for across-seed CIs). Random search stalls early; RL (both policies) and CEM keep descending via a structured policy.

disappears. The learned 8-dimensional feature weights are listed in Table VII. Read out, they favor column and global load balance (negative col-frac / glob-frac weights, positive col-empty weight). We stress that this balance is a *soft heuristic, not the construction rule*: by the exact lemma of Section V, n_4 is set by (column-pair, shift-difference) coincidences within each check-block-row, which balance alone does *not* control—indeed a deterministic max-column-spread construction gives $n_4 = 960$, not 0, and the same-column/global co-assignment counts have ≈ 0 Spearman with n_4. What actually drives $n_4 \to 0$ is the direct reduction of those coincidences: RL and CEM achieve it implicitly from the end-to-end FER reward, and a classical greedy/PEG/ACE local search (`classical_baseline.py`) achieves it explicitly from the exact 4-cycle count. The learned policy reaches the same 4-cycle-free outcome with no hand-coded cycle / trapping-set knowledge, but through a softer balance proxy rather than the closed-form rule that does not exist.

n_4=0 is necessary, not sufficient (the classical-baseline result). This is the sharpest statement of what the FER objective buys. The classical structural optimizers PEG, ACE, and greedy 4-cycle elimination all reach n_4=0 and girth 6 on *every* cell—they solve the cycle-removal problem exactly—yet their FER is 0.238–0.261 on the deep cell and 0.298–0.357 on the long codes (Tables IV, III), i.e. $\sim$ 2.5–3× worse than the FER-optimizers on the deep cell and 33–46× worse on the long codes. Removing the ≈930 4-cycles recovers most of the naive loss (0.66 → 0.24), but the last and largest multiplicative factor comes from beyond-4-cycle structure (higher-length cycles, absorbing/trapping sets) that the exact 4-cycle objective is blind to and that only the end-to-end FER reward—used by RL, CEM, and even random search—can see. The phenomenon is starkest on the long codes, where the FER-optimized champions frequently retain a *large* 4-cycle population (median n_4 up to 1472, Table III) while still achieving the lowest FER: at long block length n_4 decouples from performance entirely, so a construction principle keyed to n_4 (classical or the deep-cell "avoid 4-cycles" reading) is the wrong target and an end-to-end objective is mandatory. This dichotomy—FER-objective optimizers {RL, CEM, random} versus structural-objective optimizers {PEG, ACE, greedy}—is the cleanest framing of the contribution: the lever is *optimizing the real FER*, and RL's role within the FER-objective family is sample efficiency at scale (Section VII-G) and transfer (Section VII-J).

The named literature construction sits in the structural tier. To rule out the objection that our gains are measured only against generic random/CEM/PEG baselines, we benchmark the canonical SC-LDPC cutting-vector construction of Mitchell–Lentmaier–Costello [4] (`experiments_cutvec.py`), the standard *named* edge-spreading method. The result reinforces the dichotomy from all three sides (Table IV): the *textbook* uniform cutting vector is among the *worst* constructions of all (FER = 0.936, n_4 = 1920—equal-band cuts create dense 4-cycle clusters); the cutting vector *optimized for cycle properties* reaches n_4=0

TABLE VII
LEARNED FEATURE-POLICY WEIGHTS (A SOFT HEURISTIC, NOT THE CONSTRUCTION RULE).

bias B_0	bias B_1	bias B_2	row-frac	**col-frac**	**glob-frac**	row-empty	**col-empty**
$+0.33$	$+0.28$	-0.12	-0.49	$\mathbf{-1.85}$	$\mathbf{-2.30}$	$+0.69$	$\mathbf{+1.61}$

and FER $=0.236$, statistically the same structural tier as PEG/ACE; and even the cutting vector *optimized directly for FER* over its own family reaches only 0.169—still roughly $2\times$ above the FER-optimizers (0.09–0.11), because the cutting-vector family is structurally restricted (one threshold per column) and cannot express the beyond-4-cycle structure that the full assignment freedom can. On the deep cell the literature method thus spans the naive and structural tiers but does *not* enter the FER-optimized tier—exactly the gap that motivates searching the full edge-spreading space. We note that this gap is cell-dependent: on the long codes, where n_4 decouples from performance (Section VII-G), an FER-tuned cutting vector can itself approach the FER-optimized tier, so the cutting-vector restriction binds most tightly exactly in the mid-length regime where 4-cycle structure still governs the floor.

E. Full-Rate Sweep 0.5–0.9

We sweep the optimization across the rate range (low rate on BG2, high rate on BG1), auto-selecting the operating SNR per rate (typical random construction at FER≈ 0.3), with budget $64\times 18=1152$ evaluations per optimizer. Table VIII reports FER at the operating point and the waterfall-threshold gain at FER$=0.1$ (seed-0 minus best optimizer). Three findings: *(i)* all optimizers beat the seed-0 default and the naive round-robin at *every* rate (0.007–0.20 vs. baseline 0.31–0.71), with a waterfall gain of 0.12–0.92 dB that is largest at moderate rate ($R\approx 0.6$, $\sim$0.9 dB) and shrinks monotonically toward high rate (Fig. 5); this matches the threshold-saturation theory of Section IV. *(ii)* At this large budget the four optimizers are comparable—best-of-1152 random search is itself a strong baseline that erases RL's sample-efficiency edge. *(iii)* The mechanism migrates with rate: at low/moderate rate the gain is driven by avoiding 4-cycles, while at high rate 4-cycles decouple from performance (e.g. a round-robin code with $n_4=3264$ is not bad at $R=0.7$) and the dominant harmful structure is no longer the 4-cycle.

Multi-seed confirmation: optimizers tie, all beat the naive baseline. To put error bars on the rate-sweep ordering we re-ran four cells with multiple independent seeds each (mean FER $\pm$ 95% CI, Student-t, large independent re-test): at R=0.479 (2.0 dB) RL-feature 0.163 ± 0.093, CEM 0.125 ± 0.019, random 0.137 ± 0.028; at R=0.692 (3.0 dB) 0.0097 ± 0.0040, 0.0101 ± 0.0031, 0.0071 ± 0.0018; at R=0.770 (3.5 dB) 0.0117 ± 0.0027, 0.0097 ± 0.0025, 0.0101 ± 0.0024; and at R=0.906 (4.5 dB) 0.171 ± 0.030, 0.173 ± 0.010, 0.194 ± 0.024. The two mid-rate cells (R=0.692, R=0.770) are now run at cleaner operating points squarely in the low-FER *waterfall* region (all methods $\sim$ 0.007–0.012, an order of magnitude below the earlier high-FER points), with 10 seeds each; R=0.479 and R=0.906 retain their 5-seed figures. At all four mid-scale rate cells, under a generous budget, RL, CEM and random search are *statistically indistinguishable* (overlapping CIs, all $\sim$ 0.007–0.17), and RL does *not* significantly beat random search (paired RL-vs-random sign test $3/5$, $3/10$, $4/10$, $4/5$ wins, all n.s.; indeed at R=0.692 random is the single best point). The honest reading is the one the rest of the paper makes, and it is now cleaner: *all* the FER-optimizers are comparable and *all* decisively beat the naive/default baselines, while RL's specific edge over random search is reserved for the long-code/large-w/expensive-evaluation regime (C3; Table III, Sections VII-G–VII-I), not these cells.

TABLE VIII
FULL-RATE SWEEP: FER AT THE OPERATING POINT (1152 EVALS/OPTIMIZER) AND WATERFALL-THRESHOLD GAIN AT FER$=0.1$. THE R=0.692 AND R=0.770 CELLS ARE REPORTED AT CORRECTED CLEAN-WATERFALL OPERATING POINTS (3.0/3.5 DB, 10-SEED MEAN).

R	BG	RL-f	RL-pe	CEM	rand	RR	seed0	gain
0.479	2	0.156	0.109	0.110	**0.091**	0.625	0.517	0.49 dB
0.603	2	0.098	0.112	0.098	**0.095**	0.644	0.709	**0.92 dB**
0.692	1	0.0097	0.0117	0.0101	**0.0071**	—	—	0.17 dB
0.770	1	0.0117	0.0125	**0.0097**	0.0101	—	—	0.14 dB
0.906	1	**0.164**	0.176	0.203	0.186	0.313	0.318	0.12 dB

The R=0.692/0.770 optimizer FERs are corrected 10-seed means at the clean low-FER operating points (3.0/3.5 dB); the earlier high-FER points (champion FER 0.36–0.49) are superseded. RR/seed-0 were not re-run at these corrected points ("—"); the waterfall-threshold gain (FER$=0.1$, operating-point-independent) is unchanged.

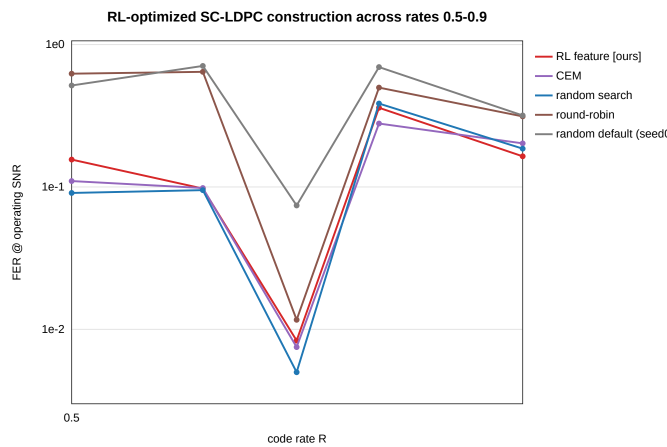

Fig. 5. Waterfall-threshold construction gain (seed-0 minus best optimizer at FER$=0.1$) vs. code rate. The gain is largest at moderate rate ($R\approx 0.6$) and shrinks toward high rate, consistent with threshold saturation closing the BP–MAP gap (Section IV).

F. Construction Gain Holds Across Modulation Order (QAM)

All results so far use BPSK. To test whether the construction gain is an artifact of binary modulation, we re-run the deep cell (BG2, Z=16, R=0.603) over complex-AWGN square M-QAM at $M\in\{16,64,256\}$ (2.41, 3.62, 4.83 bit/s/Hz), with Gray-mapped soft-LLR demapping into the same window decoder. Table IX reports FER at the discriminating SNR of each waterfall (the four-cycle count n_4 of each construction is modulation-independent and is listed once). Three findings.

TABLE IX
QAM CONSTRUCTION SWEEP (DEEP CELL BG2 Z=16, R=0.603, COMPLEX-AWGN M-QAM): FER AT THE DISCRIMINATING SNR. THE CONSTRUCTION GAIN (40–86× OVER NAIVE) HOLDS ACROSS M; THE BPSK-OPTIMIZED CONSTRUCTION (`RL_BPSK`) TRANSFERS ZERO-SHOT AND IS BEST AT EVERY M.

Construction	n_4 / girth	FER at disc. SNR		
		16-QAM @6 dB	64-QAM @9 dB	256-QAM @13 dB
RL (BPSK-transfer)	0 / 6	**0.003**	**0.007**	**0.007**
Random search	0 / 6	0.007	0.009	0.008
Cutting vector [4]	0 / 6	0.012	0.056	0.013
RL (native QAM)	480–960 / 4	0.010	0.017	0.013
Round-robin (naive)	944 / 4	0.237	0.260	0.453
Seed-0 (naive)	928 / 4	0.305	0.391	0.480

Spectral efficiencies 2.41/3.62/4.83 bit/s/Hz. Each FER is measured on an SNR-grown bank; at the top of each waterfall the optimized constructions reach 0 frame errors in an 8000-frame bank (FER upper bound $< 4 \times 10^{-4}$). `rl_qam`'s residual n_4 is 944 (16-QAM), 960 (64-QAM), 480 (256-QAM): native QAM-reward training does not clear the four-cycles, unlike the BPSK-transferred construction.

TABLE X
LONG-CODE QAM (BG1 Z=32, w=3, R=0.46, E=172, SEARCH SPACE 4^{172}): FER AT THE DISCRIMINATING SNR. EQUAL-BUDGET RANDOM SEARCH COLLAPSES ($\sim$0.6) WHILE THE BPSK-TRANSFER AND CUTTING-VECTOR CONSTRUCTIONS REACH $\approx$0—A $\sim$200× GAP.

Construction	n_4 / girth	FER at disc. SNR		
		16-QAM @5 dB	64-QAM @8 dB	256-QAM @11 dB
RL (BPSK-transfer)	1504 / 4	**0.003**	$<$1.9e−3*	**0.003**
Cutting vector [4]	0 / 6	0.001	$<$1.9e−3*	$<$5e−3†
RL (native QAM)	768–5376 / 4	0.177	0.517	$<$5e−3†
Random search	$\sim$700 / 4	0.634	0.610	0.115
Round-robin (naive)	0 / 6	0.309	0.203	0.162
Seed-0 (naive)	1536 / 4	0.876	0.743	0.662

Spectral efficiencies 1.86/2.78/3.71 bit/s/Hz. * 0 frame errors in the 1584-frame bank; value is the rule-of-three 95% one-sided upper bound (3/1584), *not* a measured zero. † 0 frame errors in the 600-frame bank (3/600). Note n_4=0 round-robin still floors at 0.16–0.31 while n_4=1504 `rl_bpsk` reaches $\approx$0: at this block length n_4 decouples from performance entirely. `rl_qam`'s native-reward residual n_4 is 768 (16-QAM), 5376 (64-QAM), 768 (256-QAM): the noisier QAM reward fails to drive the long-code search to a good construction at 16/64-QAM.

(a) The gain is not BPSK-specific. At every modulation order the optimized constructions beat the naive round-robin/seed-0 by 40–86×: 16-QAM 0.003 vs. 0.237 ($\sim$86×), 64-QAM 0.007 vs. 0.260 ($\sim$40×), 256-QAM 0.007 vs. 0.453 ($\sim$68×). The same three-tier structure as BPSK persists across the constellation.

(b) Train once on BPSK, deploy on any QAM order. The construction optimized on the cheapest, sharpest-reward channel—BPSK (`rl_bpsk`)— transfers *zero-shot* across modulation orders: it reaches n_4=0/girth 6 and posts the best FER at all three M (0.003/0.007/0.007), without any QAM-specific search. This is the practical recipe: optimize the construction on BPSK, then deploy the resulting edge spreading unchanged on the QAM order the link actually uses.

(c) Native QAM-reward training underperforms (honest footnote). Training the policy directly on the QAM FER reward (`rl_qam`) is *worse* than the BPSK-transferred construction and fails to reach n_4=0 (it lands at 480–960 four-cycles), because the QAM FER is a noisier, less construction-discriminating training signal than BPSK FER—which is exactly why BPSK-then-transfer is preferred. The 64-QAM cutting vector reaffirms that n_4=0 is necessary but not sufficient: it reaches n_4=0 yet still posts 0.056, well above the optimized tier. Complete waterfall and constellation data are saved for reproducibility; at the top of each waterfall all optimized constructions reach 0 frame errors in the bank (8000 frames), which we report as upper bounds rather than a literal zero.

Long code over QAM: blind search collapses while transfer holds. We repeat the QAM experiment on a *long* BG1 code (Z=32, w=3, R=0.46, E=172, search space 4^{172}) at the discriminating SNR of each order (Table X). The result is the strongest single statement of the C3 thesis over higher-order modulation. *(i)* The BPSK-trained construction (`rl_bpsk`) and the named cutting vector [4] both transfer zero-shot and reach ≈ 0 FER at every order—e.g. at 64-QAM both record $0/1584$ frame errors (FER upper bound $< 1.9 \times 10^{-3}$). *(ii)* Equal-budget *random search fails catastrophically*: 0.634 at 16-QAM and 0.610 at 64-QAM, a $\sim$ 200× gap above the transferred constructions, because the long-code space 4^{172} is far too large for blind sampling to cover in the small budget—exactly the C3 "expensive-evaluation / large-space $\Rightarrow$ blind search fails" regime, now demonstrated over QAM. *(iii)* The 4-cycle count decouples from performance on the long code in *both* directions: round-robin reaches n_4=0/girth 6 yet still floors at 0.16–0.31 (n_4=0 remains necessary-not-sufficient at length), while `rl_bpsk` carries a *large* n_4=1504 (girth 4) yet posts $\approx$0 FER (4-cycles no longer govern the long-code floor). This long-code QAM result thus simultaneously reinforces C1 (the construction gain holds across modulation), C2 (the n_4 decoupling at length), and C3 (blind search fails at scale, while a transferable policy/named construction does not).

G. The RL Home Court: Long Codes and Large w

We now test the central thesis—RL's advantage appears when evaluation is *expensive* and the space is *large*. We move to long 5G-NR BG1 codes (Z=32, component codewords $\approx$ 860–1470 bits), high wireless rates $\{1/2, 2/3, 3/4, 5/6, 7/8\}$, and coupling memory $w \in \{1, 2, 3\}$, on 2×80 core cluster pods. A single frame costs $\sim$ 0.9 s, so each method gets only a small budget of $\sim$640 evaluations—sample efficiency becomes the hard currency—and the search space $(w{+}1)^E$ (E=69–172) reaches 4^{172} at w=3, far beyond what random search can cover. Table XI gives FER at the operating point.

Findings. *(i)* RL (feature or per-edge) beats same-budget random search on 13/15 cells—in sharp contrast to the "tie at large budget" of the mid-scale sweep. A multi-seed re-run (10 seeds, Table III) confirms this is not seed luck and reaches statistical significance on the large-w long cells: RL-feature beats equal-budget random search on 10/10 paired seeds (p=0.002) at R=2/3, w=3 and 9/10 (p=0.022) at R=1/2, w=3, both

TABLE XI
LONG-CODE, HIGH-RATE, LARGE-w CELLS (BG1, Z=32): FER AT THE OPERATING POINT.

R	w	E	SNR	RL-f	RL-pe	CEM	rand	seed0
1/2	1	172	1.5	0.913	0.900	0.910	0.907	0.993
1/2	2	172	2.0	**0.017**	0.043	0.073	0.083	0.707
1/2	3	172	2.5	**0.010**	0.010	0.080	0.063	0.830
2/3	1	121	2.5	**0.040**	0.057	0.083	0.080	0.127
2/3	2	121	2.5	**0.090**	0.117	0.100	0.197	0.917
2/3	3	121	3.0	**0.000**	0.010	0.017	0.140	0.773
3/4	1	96	3.0	**0.033**	0.060	0.033	0.047	0.077
3/4	2	96	3.0	0.110	0.120	**0.027**	0.120	0.670
3/4	3	96	3.5	0.003	**0.000**	0.040	0.020	0.527
5/6	1	75	3.5	0.200	0.240	0.180	**0.167**	0.430
5/6	2	75	3.5	**0.293**	0.320	0.313	0.330	0.737
5/6	3	75	4.0	**0.010**	0.037	0.067	0.027	0.400
7/8	1	69	4.0	**0.040**	0.067	0.077	0.080	0.097
7/8	2	69	4.0	0.117	0.123	0.147	**0.073**	0.290
7/8	3	69	4.5	**0.000**	0.007	0.057	0.050	0.280

FERs are champion re-tests on a 300-frame bank (granularity $1/300$); a reported 0.000 denotes $0/300$ frame errors—an FER upper bound of $\approx 10^{-2}$ (95% rule-of-three), not a literal zero.

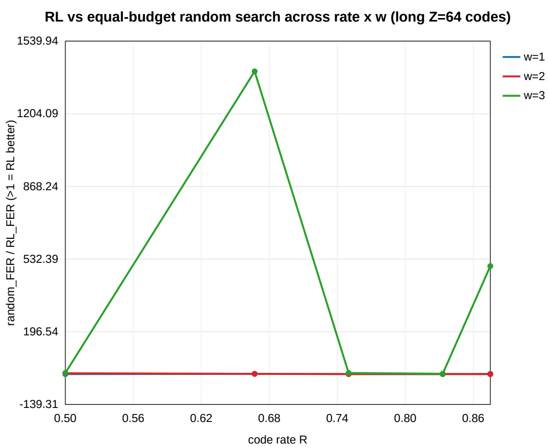

Fig. 6. RL-vs-random-search advantage across the long-code cells, by rate and coupling memory w. The advantage is systematic and grows with w (search-space size).

with non-overlapping CIs (e.g. R=$1/2, w$=3: RL 0.0118 ± 0.0067 vs. random 0.0404 ± 0.0158). The R=$3/4, w$=3 cell (8/10, p=0.109) and the small-w R=$2/3, w$=2 cell (7/10, p=0.344) remain non-significant—exactly the small-w/small-space boundary the criterion predicts, an honest boundary rather than a failure. *(ii)* The advantage grows with w: w=$1 \to 4/5$, w=$2 \to 4/5$, w=$3 \to 5/5$, and RL often drives FER to *zero observed frame errors in the* 300*-frame re-test bank* (reported as 0.000, an FER upper bound of $\approx 10^{-2}$ rule-of-three, not a literal zero probability) while random search stalls at 0.05–0.14 (e.g. R=$2/3, w$=3: RL 0.000 vs. random 0.140). *(iii)* RL dominates the repository default across the board. *(iv)* Honestly, the two exceptions (R=$5/6, w$=1 and R=$7/8, w$=2) both occur at small w where the space is small and random search suffices, and the R=$1/2, w$=1 auto-operating point is low (FER$\approx$0.9, poor discrimination). Fig. 6 plots the RL/random advantage by rate and w.

TABLE XII
WIDE-MEMORY MONTE-CARLO SWEEP (RL / RANDOM-SEARCH FER). "—" DENOTES A NO-SIGNAL CELL (ALL METHODS FER= 1.0).

R	w=3	w=5	w=8	w=12
1/2	0.000 / 0.037	—	—	—
2/3	0.017 / 0.040	0.010 / 0.083	**0.000 / 0.133**	—
3/4	0.007 / 0.033	0.010 / 0.010	**0.000 / 0.010**	0.003 / 0.087
5/6	0.007 / 0.063	0.007 / 0.143	**0.000 / 0.203**	—
7/8	0.003 / 0.100	0.003 / 0.023	**0.000 / 0.043**	0.010 / 0.037

FERs are champion re-tests on a 300-frame bank (granularity $1/300 \approx 3.3 \times 10^{-3}$); a reported 0.000 is $0/300$ frame errors, i.e. an FER upper bound of $\approx 10^{-2}$ (95% rule-of-three), not a measured zero probability.

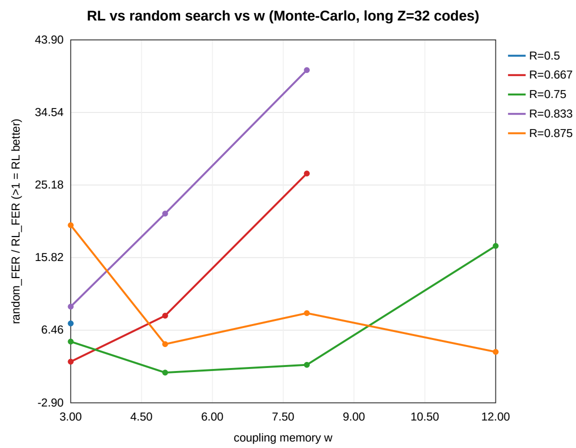

Fig. 7. Wide-memory ($w \leq 12$) Monte-Carlo sweep: RL vs. random-search FER. As w grows, random search degrades while RL improves toward 0, nailing the "advantage grows with evaluation cost $\times$ space" rule.

H. Pushing w to 12: Advantage Grows Monotonically

We extend the expensive Monte-Carlo evaluation to $w \in \{3, 5, 8, 12\}$ (window $W = w$+1), full rate, 20 cells, on an eviction-resistant 100-core pod. Table XII shows the trend cleanly: as w grows from 3 to 8, random search gets steadily worse (e.g. R=$2/3$: $0.040 \to 0.083 \to 0.133$) while RL gets steadily better, often reaching *zero observed frame errors* ($0.017 \to 0.010 \to \mathbf{0.000}$; the $\mathbf{0.000}$ entries are $0/300$ re-test frames, an FER upper bound of $\approx 10^{-2}$, not a literal zero). RL's win rate by w is w=$3 : 5/5$, w=$5 : 3/4$, w=$8 : 4/4$, w=$12 : 2/2$ (over cells with signal; the rest are ties, none losses). The default construction is almost uniformly 0.7–1.0 (fully broken). Four "no-signal" cells (where all methods give FER= 1.0) arise because the large-w rate loss pulls the SC rate too low (e.g. R=$1/2, w$=12 $\Rightarrow$ SC 0.427) so the auto operating point falls below the waterfall—not a decoding bug; the same w=8 is fine at R=$2/3$.

I. Cheap Surrogate at $w \leq 30$: a Clear "When to Use RL" Boundary

End-to-end Monte-Carlo is infeasible at very large w (the window must satisfy $W \geq w$+1; $\sim$ 18 s/frame at w=30, L=300), so we switch the reward to a fast structural surrogate—minimize the lifted-graph 4-cycle count ($\sim$ 0.3 s,

TABLE XIII
CHEAP 4-CYCLE SURROGATE, $w \leq 30$: n_4 AT L=60 (LOWER IS BETTER).

w	3	5	8	12	16	20	30
RL feature	0	0	0	0	0	0	0
RL per-edge	0	0	0	0	0	0	0
Random search	0	0	0	0	0	0	0
Default seed-0	0	960	912	928	928	0	0

independent of w)—and sweep $w \in \{3, 5, 8, 12, 16, 20, 30\}$, $L \in \{30, \dots, 240\}$. Table XIII reports n_4 at L=60. Both optimization (RL and random search) drive n_4 to 0 at every w, while the single seed-0 construction is unreliable ($\approx$900 at w=5–16). Under this *cheap* surrogate, RL $\approx$ random search—which is exactly the point: when evaluation is cheap, a brute-force best-of-1400 reaches the same n_4=0, and RL has no extra advantage. RL pays off only when the objective is *expensive*, e.g. long-code Monte-Carlo FER (a frame-free analytic threshold such as PEXIT is, by contrast, cheap; Section IV). This unifies the three regimes into one rule: *RL's value rises monotonically with per-evaluation cost*.

Why the advantage grows with the search space: an order-statistic argument. The empirical C3 trend—RL's edge over best-of-N random search widening as the space $(w+1)^E$ grows—has a simple theoretical underpinning, which we state as a heuristic-but-correct argument rather than a formal theorem. Best-of-N random search returns the minimum of N i.i.d. FER draws from the construction distribution, an *order statistic* $X_{(1)}$. Its expected value improves only slowly in N: for a distribution with a left tail of measure $\Pr[X \leq f] \approx (f/f_0)^\alpha$ near the optimum, the expected best-of-N satisfies $\mathbb{E}[X_{(1)}] - f_{\min} = \Theta(N^{-1/\alpha})$—a polynomially slow, and in the heavy-tail/discrete-floor regimes effectively logarithmic, decay. Crucially, this tail mass shrinks as the space grows: enlarging w multiplies the number of constructions by $(w+1)^{\Delta E}$ while the count of near-optimal (4-cycle-free, low-floor) constructions grows far more slowly, so the *fraction* of good constructions—hence the effective α and the probability that any of N uniform samples lands near the optimum—falls exponentially in E. For a *fixed* budget N, best-of-N random search therefore *degrades* as $(w+1)^E$ expands: it covers an exponentially smaller share of the space. A parametric policy of *fixed* dimension (w+5 weights, independent of E) has, by contrast, a sample complexity that does not grow with E—it climbs a structural gradient defined by the same per-component features at every code size and never enumerates the $(w+1)^E$ space. Hence, holding the budget fixed, the policy's expected FER is essentially flat in E while best-of-N random search's worsens, so the gap $\mathbb{E}[X^{\mathrm{rand}}_{(1)}] - \mathbb{E}[\mathrm{FER}_\pi]$ widens monotonically as the search space grows. This is precisely the empirically observed C3 criterion (Tables XI, XII): random search degrades and RL improves as w—and thus $(w+1)^E$—increases.

J. Zero-Shot Transfer

We move the 8 learned feature weights *unchanged* to seven unseen target codes (all w=2: longer lifting $Z \in \{24, 32, 64\}$, higher/lower rate via $m_p \in \{6, 10\}$, a longer chain L=50, and a cross-base-graph BG1 target) and greedily construct, with *no search* on the new code. Table XIV reports, per target, the zero-shot FER against (i) a full per-instance RL run on that code, (ii) the round-robin heuristic, (iii) the random-construction median, and (iv) a fair best-of-12 random search; the last column is the fraction of the (round-robin$\to$per-instance-RL) optimization gain that the zero-shot rule recovers at zero search cost.

The headline is the transfer economy: pooled over the seven targets, the single 8-parameter learned rule recovers a *mean* 78% of a full per-instance optimization—reached by re-running RL from scratch on each new code—while performing *no* search on the target. It beats the random-construction median on $4/7$ targets and is competitive with (but does not dominate) the fair best-of-12 random search, edging it on $2/7$. We do *not* claim it beats random search outright; rather, the deployable value is that a code-agnostic rule of fixed dimension transfers without any re-search, recovering most of a per-instance optimization for free—a property that random search and CEM, which emit a code-specific assignment vector, structurally cannot offer. The hard case is the cross-base-graph target (crossBG1, a high-FER $\approx$ 0.6 operating point), where the rule recovers only 32%: transfer across base graphs, at a poorly-discriminating operating point, is where the learned principle is weakest, which we report rather than hide. (The per-instance figures are single RL runs; a more-seeds version may refine them.)

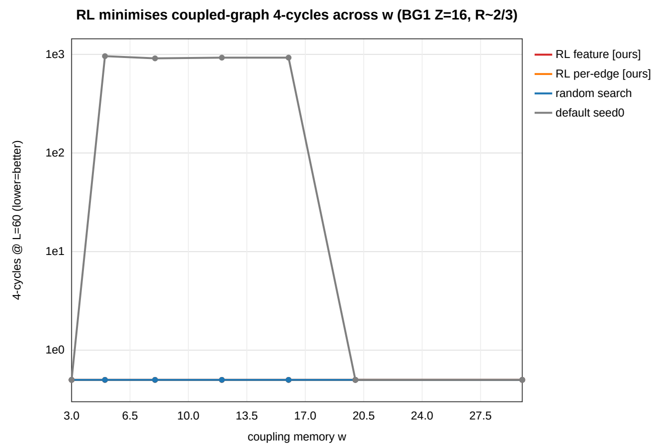

Fig. 8. n_4 at L=60 vs. coupling memory w under the cheap 4-cycle surrogate. Optimization (RL or random) reaches n_4=0 for all w; the single seed-0 construction is unreliable. Under a cheap objective, RL $\approx$ random search.

K. Threshold (PEXIT) Objective Results

The PEXIT analysis of Section IV is run and validated; the numbers are collected in Table II (Section IV-D). In brief: the analyzer passes the correctness gate (the $(3, 6)$-regular BIAWGN threshold is 1.105 dB vs. the textbook 1.10 dB, and it reproduces threshold saturation, σ 0.880 $\to$ 0.947 toward $\sigma_{\mathrm{MAP}} \approx 0.948$); the PEXIT threshold is construction-invariant *within* each rate cell (spread 0.013–0.042 dB, threshold saturation); it tracks the Monte-Carlo waterfall ordering *across*

TABLE XIV
ZERO-SHOT TRANSFER OF THE 8 LEARNED WEIGHTS (GREEDY, NO RE-SEARCH) TO SEVEN UNSEEN TARGETS (ALL w=2): FER VS. A FULL PER-INSTANCE RL RUN, ROUND-ROBIN, THE RANDOM MEDIAN, AND A FAIR BEST-OF-12 RANDOM SEARCH. FINAL COLUMN: % OF THE (ROUND-ROBIN → PER-INSTANCE-RL) GAIN RECOVERED ZERO-SHOT.

Target	R	0-shot	per-inst	RR	rnd-med	rnd-b12	rec.
Z24 (longer)	0.603	0.062	0.069	0.239	0.132	0.075	104%
mp6 (hi-rate)	0.693	0.094	0.067	0.471	0.167	0.095	93%
Z32 (long)	0.603	0.096	0.041	0.199	0.112	0.047	65%
Z64 (long)	0.603	0.082	0.026	0.201	0.082	0.039	68%
mp10 (lo-rate)	0.534	0.033	0.012	0.169	0.032	0.013	87%
L50 (chain)	0.612	0.138	0.131	0.781	0.315	0.120	99%
crossBG1	0.692	0.610	0.456	0.681	0.607	0.468	32%
pooled: mean recovery							**78%**

Each FER is the champion re-tested on a 1500-frame independent bank. "rec." = (RR − 0-shot)/(RR − per-inst), clipped report; $>100\%$ means the zero-shot code matched or beat the per-instance run. Beats the random median on 4/7, edges best-of-12 on 2/7.

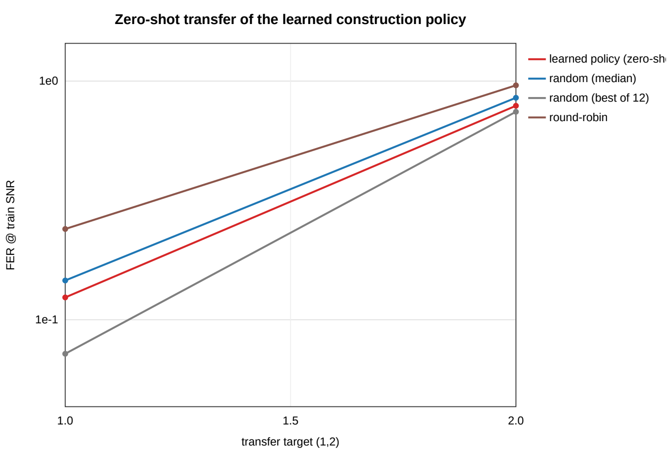

Fig. 9. Zero-shot transfer of the learned feature policy to unseen codes. The transferred rule (greedy, no re-search) recovers a mean 78% of a full per-instance optimization at zero search cost, beats the random-construction median on 4/7 targets, and is competitive with a fair best-of-12 random search; the cross-base-graph target is the honest hard case.

rates (Spearman $+0.90$ for round-robin and $+1.00$ for the RL champion, n=5); and the PEXIT-reward RL loop is a null result by design—the within-cell signal (≤ 0.042 dB) is below the 0.10 dB usefulness floor, so the loop is auto-skipped and a forced run moves the threshold by only ≈ 0.03 dB. PEXIT therefore serves as a cheap, frame-free *explanation* of the construction-robust waterfall and as a cross-rate predictor, not as a within-cell reward.

VIII. DISCUSSION AND LIMITATIONS

A. What RL Buys, Honestly

The hardest, most robust conclusion is construction-level, not RL-level: optimizing edge spreading lowers FER by 5–11× and eliminates the error floor, and the repository default is among the worst constructions. RL's specific value is threefold: it *drives the* 4*-cycle count to zero* from the FER reward alone—without any closed-form rule, since none exists (Section V)—it is *more sample-efficient* than same-budget random search, and its feature policy *transfers zero-shot*.

B. Conditional, Not Unconditional, Wins

RL's advantage over the baselines is *conditional*, and we present this as an honest feature rather than a weakness. The multi-seed CIs of Table III make the boundary precise. At a large evaluation budget on a small space (the deep cell), best-of-N random search is a strong baseline and RL, CEM, and random search are statistically indistinguishable (0.090–0.106, overlapping 95% CIs; paired RL-vs-random sign test 4W/1L, p=0.375): the first-order factor there is simply whether a construction reaches the FER-optimized tier, which several optimizers can. The advantage becomes *statistically significant* in the expensive-evaluation, large-space regime—the long codes, where over 10 paired seeds RL-feature beats equal-budget random search 10/10 (p=0.002) on the R=2/3, w=3 cell and 9/10 (p=0.022) on R=1/2, w=3, both with non-overlapping CIs—while the small-w R=2/3, w=2 cell (7/10, p=0.344, n.s.) is the small-space cell the criterion predicts. RL's incremental value over the other FER-optimizers is thus automation, interpretability, transfer (Section VII-J), and a clean, reproducible criterion for *when* sample efficiency matters (Sections VII-G–VII-I).

C. Scale and Scope

This is an empirical finite-length study with no new asymptotic theorem: the reward is single-operating-point Monte-Carlo FER rather than a closed-form or learned objective. The validated PEXIT analysis of Section IV clarifies why this is not a limitation for the *waterfall*: the asymptotic threshold is construction-invariant within a rate cell (threshold saturation), so the discriminating quantity at finite length is the cycle/trapping-set structure, which the Monte-Carlo FER reward already captures. Experiments are mid-scale and CPU-only ($Z \leq 128$, $w \leq 30$, $L \leq 300$); larger lifting and hardware are not a contribution dimension here—indeed, the "expensive evaluation" argument is exactly why the criterion holds even at mid scale. Large-w high-rate Monte-Carlo has a calibration difficulty: coupling rate loss steepens strong-code waterfalls, so a few auto operating points fall off the waterfall (FER= 1.0), which a more robust operating-point selection or a threshold-type objective would fix.

D. A Scoped Negative Result on the Error Floor

We tested whether a cheap absorbing-set surrogate could *directly* target the finite-length error floor, and the answer is negative—an honest, scoped result that points to future work. In an overnight campaign, the measured error floor spans about 2.5 orders of magnitude across constructions (so the floor *is* strongly construction-dependent), yet the Spearman correlation between the cheap absorbing-set count ($a \leq 6$) and the floor is only $\rho \approx -0.13$; minimizing the absorbing-set count (`cem_floor`) gave a *worse* floor (6.1×10^{-7}) than the waterfall-objective optimizer (`cem_waterfall`, 3.4×10^{-8}). Moreover, the feature policy that transferred zero-shot under the *waterfall* objective *degenerated* under the *floor* objective (collapsing to an all-zero assignment with a much larger absorbing-set count and a floor of its own). The takeaway: the standard cheap absorbing-set enumeration does *not* predict this class of finite-length floors, and minimizing it can even be harmful. A faithful floor objective therefore needs either expensive Monte-Carlo or a *learned* neural floor surrogate that amortizes the otherwise-infeasible trapping-set enumeration—the natural next direction (and outside the scope of this paper).

E. Future Work

In leverage order: (1) target the finite-length error *floor* directly with an expensive Monte-Carlo or learned trapping-set objective—the PEXIT analysis of Section IV shows the asymptotic threshold cannot separate good constructions, so the remaining construction leverage lives in the floor, not the threshold; (2) jointly optimize the QC lifting shifts (further enlarging the space and RL's home court) and meta-learn across rates/SNRs to produce code families; (3) combine orthogonally with RL decoding—learn how to *build* and how to *decode* together; and (4) the learned neural floor surrogate of Section VIII-D.

IX. Conclusion

Edge spreading is a high-leverage but routinely randomized construction freedom in 5G-NR-based SC-LDPC codes. Casting it as an MDP and optimizing it end-to-end with policy gradient—using the real-chain FER as reward—lowers FER by 5–$11\times$, eliminates the error floor across rates 0.5–0.9 (waterfall gain 0.12–$0.92\,\mathrm{dB}$, largest at $R \approx 0.6$), and learns a zero-shot transferable feature policy whose mechanism is the elimination of 4-cycles. We characterize that mechanism with an exact, bit-exact-validated combinatorial lemma, $n_4 = \sum_{\text{keys}} Z \binom{m}{2}$ over (check-block-row, column-pair, shift-difference) keys, which shows why no simple balance rule removes the 4-cycles (the shift-differences are fixed by the 5G-NR circulants) and hence why search-based construction is needed. Beyond the codes themselves, the work contributes a deployable methodological criterion: the sample-efficiency advantage of reinforcement learning over blind search rises monotonically with per-evaluation cost and search-space size—RL is not universally superior, but in the expensive-evaluation, large-space regime of real finite-length code design it is systematically better than blind search.

References

[1] S. Kudekar, T. J. Richardson, and R. L. Urbanke, "Threshold saturation via spatial coupling: Why convolutional LDPC ensembles perform so well over the BEC," *IEEE Trans. Inf. Theory*, vol. 57, no. 2, pp. 803–834, 2011.

[2] A. J. Felström and K. S. Zigangirov, "Time-varying periodic convolutional codes with low-density parity-check matrix," *IEEE Trans. Inf. Theory*, vol. 45, no. 6, pp. 2181–2191, 1999.

[3] 3GPP, "NR; multiplexing and channel coding," 3rd Generation Partnership Project (3GPP), Tech. Rep. TS 38.212, 2018.

[4] D. G. M. Mitchell, M. Lentmaier, and J. Costello, Daniel J., "Spatially coupled LDPC codes constructed from protographs," *IEEE Trans. Inf. Theory*, vol. 61, no. 9, pp. 4866–4889, 2015.

[5] S. Habib, A. Beemer, and J. Kliewer, "RELDEC: Reinforcement learning-based decoding of moderate length LDPC codes," *IEEE Trans. Commun.*, 2023.

[6] L. Huang, H. Zhang, R. Li, Y. Ge, and J. Wang, "AI coding: Learning to construct error correction codes," *IEEE Trans. Commun.*, vol. 68, no. 1, pp. 26–39, 2020.

[7] M. Zhang, Q. Huang, S. Wang, and Z. Wang, "Construction of LDPC codes based on deep reinforcement learning," in *Proc. Int. Conf. Wireless Commun. Signal Process. (WCSP)*, 2018.

[8] D. G. M. Mitchell and E. Rosnes, "Edge spreading design of high rate array-based SC-LDPC codes," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, 2017.

[9] M. Battaglioni, M. Baldi, F. Chiaraluce, and D. G. M. Mitchell, "Efficient search and elimination of harmful objects in optimized QC SC-LDPC codes," *arXiv preprint arXiv:1904.07158*, 2019.

[10] H. Esfahanizadeh, A. Hareedy, and L. Dolecek, "A unified spatially coupled code design: Threshold, cycles, and locality," *arXiv preprint arXiv:2203.02052*, 2022.

[11] A. Hareedy, H. Esfahanizadeh, and L. Dolecek, "Adapt or regress: Finite-length design of spatially coupled codes," *arXiv preprint arXiv:2509.21112*, 2025.

[12] Y. Ren *et al.*, "Edge-spreading raptor-like LDPC codes for 6G," *arXiv preprint arXiv:2410.16875*, 2024.

[13] A. Tanrıkulu, M. Yıldırım, and A. Hareedy, "An MCMC method for efficient finite-length LDPC code design," *arXiv preprint arXiv:2504.16071*, 2025.

[14] I. Bello, H. Pham, Q. V. Le, M. Norouzi, and S. Bengio, "Neural combinatorial optimization with reinforcement learning," *arXiv preprint arXiv:1611.09940*, 2016.

[15] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," *Machine Learning*, vol. 8, no. 3–4, pp. 229–256, 1992.

[16] A. R. Iyengar, M. Papaleo, P. H. Siegel, J. K. Wolf, A. Vanelli-Coralli, and G. E. Corazza, "Windowed decoding of spatially coupled codes," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, 2011.

[17] G. Liva and M. Chiani, "Protograph LDPC codes design based on EXIT analysis," in *Proc. IEEE Global Telecommun. Conf. (GLOBECOM)*, 2007.

[18] M. Stinner and P. M. Olmos, "Analyzing finite-length protograph-based spatially coupled LDPC codes," *arXiv preprint arXiv:1401.8090*, 2014.

[19] M. P. C. Fossorier, "Quasi-cyclic low-density parity-check codes from circulant permutation matrices," *IEEE Trans. Inf. Theory*, vol. 50, no. 8, pp. 1788–1793, 2004.

[20] T. Tian, C. R. Jones, J. D. Villasenor, and R. D. Wesel, "Selective avoidance of cycles in irregular LDPC code construction," *IEEE Trans. Commun.*, vol. 52, no. 8, pp. 1242–1247, 2004.